

Assignment: Classification of radar returns from the ionosphere

Due: May 19, 2026, 23:00 CET.

Machine Learning Course - Master in Space Technologies and Business - University of Luxembourg

Summer Semester 2026

Instructions on submitting your answers

Read all instructions in this section thoroughly.

Collaboration: **This assignment must be completed individually.** You may discuss the homework at a high level to better understand the problems and underlying concepts, but you must not share or exchange solutions, code, or detailed answers with others.

Posting any part of your solution online is not permitted, including after the course has ended. Likewise, please do not share or upload this assignment document to the internet.

Citing Your Sources: Any sources of help that you consult while completing this assignment (other students, textbooks, websites, etc.) ***MUST*** be noted in your submitted PDF document.

Declare the use of generative AI and for what purpose.

Formatting: This assignment consists of two parts: answering the different questions and implementing different learning algorithms.

- you must write up your solution electronically and submit it as a single PDF document, which you will submit through **Moodle**. Handwritten or paper copies are not accepted.
- Your implementation must be in Python.

 **Submitting Your Solution:** You will be submitting only the following files:

1. UNI-ID.pdf (a PDF with your answers + explanations).
 2. Dataset (if modified).
 3. **Python code (.ipynb or .py).**
-

Objective

In this assignment, you will:

- Apply **Machine Learning and Deep Learning using Python**
- Follow **structured workflows** for both approaches
- Compare their performance on the same dataset

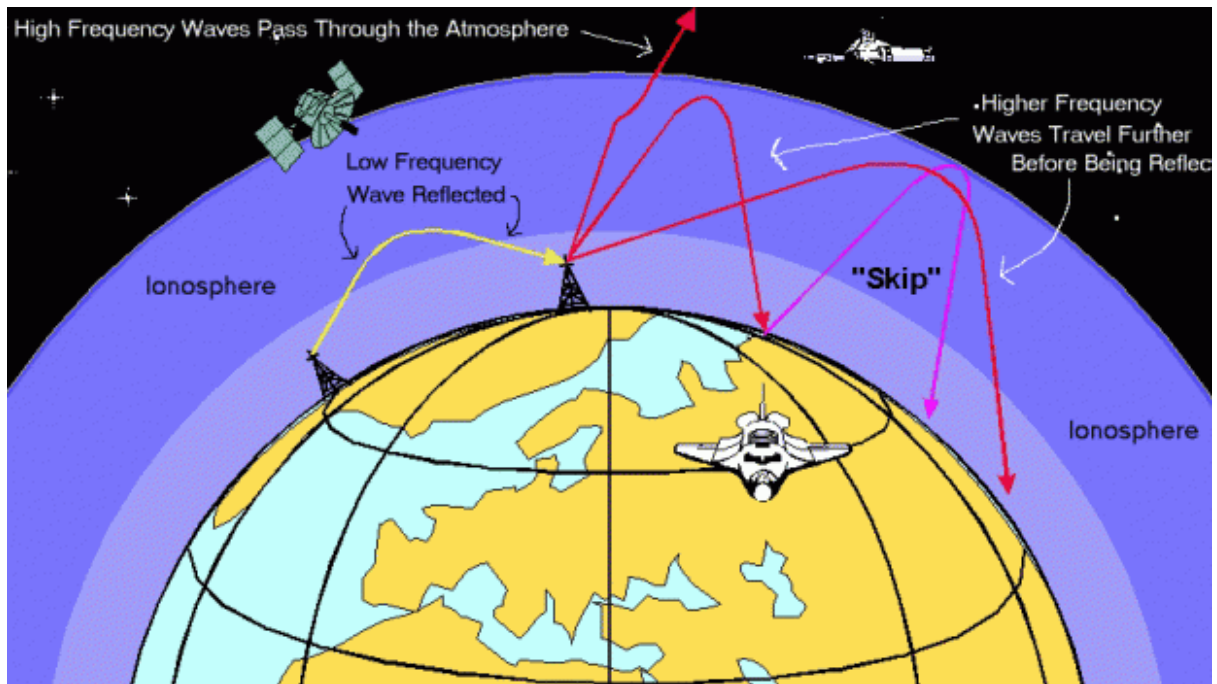


Figure 1 Source: <https://www.swpc.noaa.gov/phenomena/ionosphere>

In this assignment, you will implement some learning models to classify radar returns from the ionosphere as either suitable for further analysis (“good”) or not (“bad”).

Dataset:

Title: *Johns Hopkins University Ionosphere database:*

Description: This dataset includes radar data was collected by a system in Goose Bay, Labrador. This system consists of a phased array of 16 high-frequency antennas with a total transmitted power on the order of 6.4 kilowatts. The targets were free electrons in the ionosphere. "Good" radar returns are those showing evidence of some type of structure in the ionosphere. "Bad" returns are those that do not; their signals pass through the ionosphere.

Source: <https://archive.ics.uci.edu/dataset/52/ionosphere>

Use the provided **ionosphere dataset** (same as original assignment).

Task 1: Understand and Prepare the Data

1.1 Dataset Exploration

- Load dataset using Python
- Report:
 - Size (MB)
 - Number of samples
 - Number of features
 - Data types

- Missing values
- Basic statistics (describe())

1.2 Data Understanding

Explain what makes a radar return good or bad? Which is priority to detect more accurately? (≤100 words)

1.3 Data Preprocessing

In your own words, describe the purpose and key steps involved in each of the following data preprocessing activities: *Limit your answer to approximately 50 words per item*

- Data cleaning
- Feature scaling (e.g., StandardScaler)
- Train-test split



Task 2: Machine Learning Workflow

You must follow this **step-by-step ML pipeline** using Python (e.g., scikit-learn).



ML Workflow (must be clearly shown in your report)

1. Load and split data
2. Preprocess features (e.g., scaling/encoding, if needed)
3. Choose model (Logistic Regression)
4. Train the model
5. Make predictions
6. Evaluate the model

2.1 Model Training

Train a **Logistic Regression model**

2.2 Evaluation Methods

- Train/test split
- K-fold cross-validation (*choose a suitable k value and justify your choice*).

2.3 Metrics

Report the following metrics for “good” and “bad”, separately:

- Accuracy
- Precision

- Recall
- F1-score
- Confusion matrix

2.4 Analysis

- Compare evaluation methods (≤ 100 words)
 - Suggest improvements (≤ 100 words)
 - Implement a baseline model that predicts the majority class and compare its performance to a Logistic Regression model (≤ 100 words)
-



Task 3: Deep Learning Workflow

Use PyTorch.



DL Workflow (must be clearly shown in your report)

1. Load and preprocess data
 2. Convert data to tensors
 3. Define neural network architecture
 4. Define loss function and optimizer
 5. Train model (epochs, loop)
 6. Evaluate model
 7. Monitor performance (loss curves)
-

3.1 Build a Neural Network

- Input layer = number of features
- At least **1 hidden layer**
- Output layer = binary classification

3.2 Training

- Train for multiple epochs
- Show training loss

3.3 Evaluation

Report the same metrics as ML for “good” and “bad” separately.

- Accuracy, Precision, Recall, F1-score
-



Task 4: ML vs Deep Learning Comparison

4.1 Results Table

Create a table comparing (use weighted average):

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression				
Neural Network				

4.2 Comparison Questions

Answer the following:

- 1. Which model performs better? Why?** (≤ 150 words)
- 2. Which model is more computationally expensive? Why?** (≤ 100 words)
- 3. When would you prefer ML over DL and vice versa?** (≤ 150 words)



Task 5: Improving the Models

5.1 ML Improvements

Try another mode (e.g., random forest). Report the results in a table using the metrics mentioned above.

5.2 DL Improvements

Try adding more layers. Report the results in a table using the metrics mentioned above.

5.3 Reflection (≤ 150 words)

Which improvements helped most?